

TP 4 Java EE

Migration vers Maven et JPA/Hibernate

Gestion de dépendances & ORM

4ème GL – École Polytechnique de Sousse

Saoudi Haythem

Année universitaire 2025–2026

Table des matières

1	Introduction à Apache Maven	2
1.1	Pourquoi Maven ?	2
1.2	Le fichier pom.xml	2
1.3	Le dépôt Maven local et central	3
2	Migration du projet vers Maven	4
2.1	Conversion du projet en projet Maven	4
2.2	Suppression des anciens fichiers JAR	4
2.3	Ajout des dépendances Maven	4
3	Introduction à JPA/Hibernate	6
3.1	Le patron ORM (Object Relational Mapping)	6
3.2	Annotations JPA sur la classe Produit	6
3.3	Configuration de persistence.xml	9
4	Implémentation de la couche DAO avec JPA	11
4.1	EntityManagerFactory et EntityManager	11
4.2	Méthodes CRUD avec JPA	11
4.3	Mise à jour de la Servlet	14
5	Vérification et Tests	14
5.1	Génération automatique de la table	15
5.2	Tests des opérations CRUD	15

1 Introduction à Apache Maven

Apache Maven est un outil de build et de gestion de projets Java qui automatise la compilation, les tests, le packaging et la gestion des dépendances via un fichier de configuration central `pom.xml`.

Objectif de la section

Comprendre le rôle de Maven, maîtriser le fichier `pom.xml` et migrer un projet Web dynamique Eclipse en projet Maven.

1.1 Pourquoi Maven ?

Problèmes sans Maven :

- Gestion manuelle des fichiers `.jar` (copie dans `WEB-INF/lib`)
- Risque d'incompatibilité de versions entre bibliothèques
- Pas de centralisation : chaque développeur télécharge les mêmes fichiers
- Absence d'automatisation du cycle de vie (compilation, tests, déploiement)

Avantages de Maven :

- Gestion déclarative des dépendances dans `pom.xml`
- Téléchargement automatique depuis le dépôt central (<https://mvnrepository.com>)
- Standardisation de la structure de projet
- Cycle de vie unifié : `validate` → `compile` → `test` → `package` → `install` → `deploy`

1.2 Le fichier `pom.xml`

Définition : Project Object Model (POM)

Le fichier `pom.xml` est le cœur d'un projet Maven. Il décrit le projet, ses dépendances, ses plugins, ainsi que la configuration liée au cycle de vie du build. Chaque projet Maven est identifié de façon unique par trois coordonnées :

- `groupId` : identifiant de l'organisation (ex. `com.poly.jee`)
- `artifactId` : nom du projet/module (ex. `gestion-produits`)
- `version` : version du projet (ex. `1.0-SNAPSHOT`)

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0">
2   <modelVersion>4.0.0</modelVersion>
3
4   <groupId>com.poly.jee</groupId>
5   <artifactId>gestion-produits</artifactId>
6   <version>1.0-SNAPSHOT</version>
7   <packaging>war</packaging>
8   <name>Gestion Produits JEE</name>
```

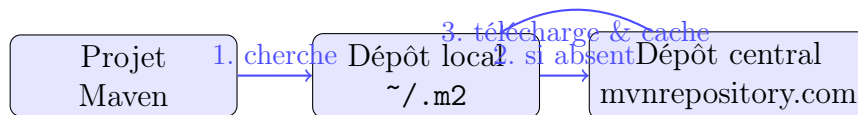
```
9
10 <dependencies>
11   <!-- Les dependances vont ici -->
12 </dependencies>
13 </project>
```

Listing 1 – Structure minimale d'un pom.xml

1.3 Le dépôt Maven local et central

Étape 1 : Fonctionnement des dépôts Maven

Comprendre comment Maven résout les dépendances :



Remarque

Maven vérifie d'abord le dépôt local (`~/.m2/repository`). Si la dépendance est absente, il la télécharge depuis le dépôt central et la met en cache localement pour les prochaines utilisations.

2 Migration du projet vers Maven

Objectif de la section

Convertir le projet Web dynamique Eclipse existant en projet Maven et configurer les trois dépendances nécessaires : MySQL Connector, JSTL et Hibernate Core.

2.1 Conversion du projet en projet Maven

Étape 1 : Conversion dans Eclipse

Faites un clic droit sur le nom du projet dans Eclipse puis sélectionnez **Configure** → **Convert to Maven Project**.

Attention

Cette opération est irréversible. Assurez-vous que votre projet compile et fonctionne correctement avant la conversion. Pensez également à faire une sauvegarde du projet.

Étape 2 : Configuration du groupId

Dans la boîte de dialogue qui apparaît, modifiez le **groupId** :

- **GroupId** : `com.poly.jee`
- **ArtifactId** : nom du projet (conservé automatiquement)
- **Version** : `0.0.1-SNAPSHOT`

Validez avec **Finish**.

Remarque

Après la conversion, le projet garde la même structure mais un fichier `pom.xml` est ajouté à la racine. Le dossier `WEB-INF/lib` sera géré automatiquement par Maven.

2.2 Suppression des anciens fichiers JAR

Étape 3 : Nettoyage du dossier lib

Avant d'ajouter les dépendances Maven, supprimez tout le contenu de :

`src/main/webapp/WEB-INF/lib`

Maven prendra le relais pour fournir toutes les bibliothèques nécessaires.

2.3 Ajout des dépendances Maven

Notre application nécessite trois dépendances : **MySQL Connector**, **JSTL** et **Hibernate Core**. Recherchez-les sur <https://mvnrepository.com> et ajoutez-les dans la

balise `<dependencies>` du `pom.xml` :

Étape 4 : Ajout des dépendances dans `pom.xml`

Copiez le bloc `<dependencies>` suivant dans votre `pom.xml` :

```
1 <dependencies>
2
3   <!-- 1. MySQL Connector Java -->
4   <dependency>
5       <groupId>mysql</groupId>
6       <artifactId>mysql-connector-java</artifactId>
7       <version>5.1.38</version>
8   </dependency>
9
10  <!-- 2. JSTL (JavaServer Pages Standard Tag Library) -->
11  <dependency>
12      <groupId>javax.servlet</groupId>
13      <artifactId>jstl</artifactId>
14      <version>1.2</version>
15  </dependency>
16
17  <!-- 3. Hibernate Core (ORM) -->
18  <dependency>
19      <groupId>org.hibernate</groupId>
20      <artifactId>hibernate-core</artifactId>
21      <version>5.6.5.Final</version>
22  </dependency>
23
24  <!-- Servlet API (fournie par le serveur, scope provided)
25  <dependency>
26      <groupId>javax.servlet</groupId>
27      <artifactId>javax.servlet-api</artifactId>
28      <version>4.0.1</version>
29      <scope>provided</scope>
30  </dependency> -->
31 </dependencies>
```

Listing 2 – `pom.xml` complet avec les 3 dépendances

Remarque

Dès que vous enregistrez le `pom.xml`, Maven télécharge automatiquement les dépendances déclarées (connexion Internet requise) et les ajoute au *Build Path* du projet.

3 Introduction à JPA/Hibernate

JPA (Java Persistence API) est la spécification standard Java EE pour la persistance des données. Elle définit une API uniforme pour mapper les objets Java vers les tables d'une base de données relationnelle. **Hibernate** est l'implémentation la plus répandue de cette spécification.

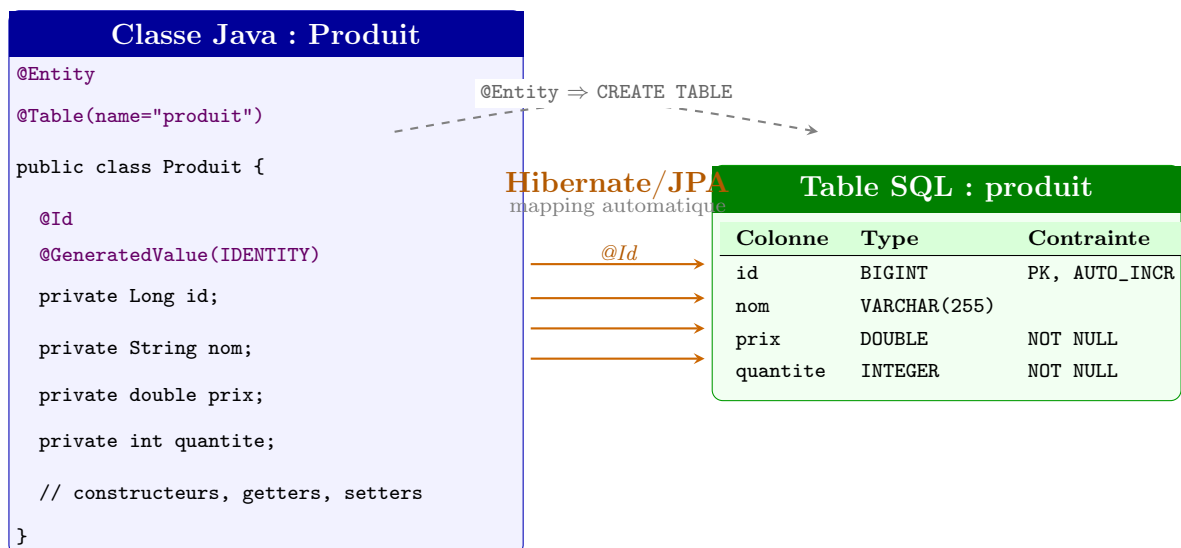
Objectif de la section

Comprendre le patron ORM, remplacer la couche DAO JDBC par JPA/Hibernate, configurer `persistence.xml` et implémenter les opérations CRUD via l'EntityManager.

3.1 Le patron ORM (Object Relational Mapping)

Définition : ORM — Object Relational Mapping

Un ORM est un framework qui assure la correspondance automatique entre les **objets Java** (entités) et les **tables de la base de données**, éliminant la majorité du code SQL manuel et rendant le code indépendant du SGBD.



Avantages de JPA/Hibernate :

- Élimination du code SQL répétitif (CRUD généré automatiquement)
- Indépendance du SGBD (MySQL, PostgreSQL, Oracle...)
- Gestion automatique des transactions
- Création automatique des tables à partir des entités (`hbm2ddl`)

3.2 Annotations JPA sur la classe Produit

Étape 1 : Transformation de la classe Produit en entité JPA

Ajoutez les annotations JPA sur la classe `Produit` pour la rendre persistante :

```
1 package com.poly.jee.entities;
2
3 import javax.persistence.Entity;
4 import javax.persistence.GeneratedValue;
5 import javax.persistence.GenerationType;
6 import javax.persistence.Id;
7 import javax.persistence.Table;
8
9 @Entity // Declare cette classe comme entite JPA
10 @Table(name = "produit") // Nom de la table en BD (optionnel)
11 public class Produit {
12
13     @Id // Cle primaire
14     @GeneratedValue(strategy = GenerationType.IDENTITY) //
15     AUTO_INCREMENT
16     private Long id;
17
18     private String nom;
19     private double prix;
20     private int quantite;
21
22     // Constructeur vide obligatoire pour JPA
23     public Produit() {}
24
25     public Produit(Long id, String nom, double prix, int
26     quantite) {
27         this.id = id;
28         this.nom = nom;
29         this.prix = prix;
30         this.quantite = quantite;
31     }
32
33     // Getters et Setters
34     public Long getId() { return id; }
35     public void setId(Long id) { this.id = id; }
36     public String getNom() { return nom; }
37     public void setNom(String nom) { this.nom = nom; }
38     public double getPrix() { return prix; }
39     public void setPrix(double prix) { this.prix = prix; }
40     public int getQuantite() { return quantite; }
41     public void setQuantite(int quantite) { this.quantite =
42     quantite; }
43 }
```

Listing 3 – Classe Produit.java avec annotations JPA

Remarque**Correspondances annotation/SQL :**

- `@Entity` → la classe correspond à une table
- `@Id` → le champ est la clé primaire (PRIMARY KEY)
- `@GeneratedValue(IDENTITY)` → la valeur est auto-incrémentée (AUTO_INCREMENT)
- Les autres attributs → colonnes de la table (nom identique par défaut)

3.3 Configuration de persistence.xml

Étape 2 : Création de persistence.xml

Créez l'arborescence `src/main/resources/META-INF/` et placez-y le fichier `persistence.xml` :

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <persistence xmlns="https://jakarta.ee/xml/ns/persistence"
3   version="3.0">
4   <persistence-unit name="MonUniteDePersistence"
5     transaction-type="RESOURCE_LOCAL">
6
7     <provider>org.hibernate.jpa.HibernatePersistenceProvider</
8       provider>
9
10    <properties>
11      <!-- 1. Pilote JDBC MySQL Connector/J 8 -->
12      <property name="jakarta.persistence.jdbc.driver"
13        value="com.mysql.cj.jdbc.Driver"/>
14
15      <!-- 2. URL de connexion -->
16      <property name="jakarta.persistence.jdbc.url"
17        value="jdbc:mysql://localhost:3306/gestioncommv2
18          "/>
19
20      <!-- 3. Identifiants MySQL -->
21      <property name="jakarta.persistence.jdbc.user" value="
22        root"/>
23      <property name="jakarta.persistence.jdbc.password" value="
24        "/>
25
26      <!-- 4. Dialecte Hibernate (MySQL generique, Hibernate 6+)
27        -->
28      <property name="hibernate.dialect"
29        value="org.hibernate.dialect.MySQLDialect"/>
30
31      <!-- 5. Afficher le SQL genere -->
32      <property name="hibernate.show_sql" value="true"/>
33      <property name="hibernate.format_sql" value="true"/>
34
35      <!-- 6. Gestion du schema: update (dev) / validate / none
36        (prod) -->
37      <property name="hibernate.hbm2ddl.auto" value="update"/>
38    </properties>
39  </persistence-unit>
40 </persistence>

```

Listing 4 – persistence.xml – Configuration JPA/Hibernate

Remarque

Valeurs possibles pour `hbm2ddl.auto` :

- `create` : recrée les tables à chaque démarrage (perte de données!)
- `create-drop` : crée au démarrage, supprime à l'arrêt
- `update` : crée si absent, met à jour si structure modifiée (**recommandé en dev**)
- `validate` : valide sans modifier (recommandé en production)
- `none` : désactivé

Attention

Base de données : Créez une base de données vide nommée `gestioncommv2` dans MySQL avant d'exécuter l'application. Hibernate créera automatiquement la table `produit` au premier lancement.

4 Implémentation de la couche DAO avec JPA

Objectif de la section

Créer la classe `GestionProduitImpJpa` qui implémente l'interface `IGestionProduit` en utilisant l'API JPA (`EntityManager`) pour toutes les opérations CRUD.

4.1 EntityManagerFactory et EntityManager

Définition : EntityManagerFactory

L'`EntityManagerFactory` est une fabrique JPA **coûteuse à initialiser** (chargement du pilote, lecture de `persistence.xml`, établissement du pool de connexions). Elle doit être instanciée **une seule fois** au démarrage de l'application via `Persistence.createEntityManagerFactory("nom-unite")`, puis réutilisée pour créer des `EntityManager`.

Définition : EntityManager

L'`EntityManager` est l'objet JPA central qui gère le cycle de vie des entités. Il fournit toutes les méthodes CRUD : `persist()` (ajout), `find()` (recherche), `merge()` (modification), `remove()` (suppression). Il est **léger** et créé à partir de l'usine via `emf.createEntityManager()`.

Étape 1 : Implémentation de toutes les méthodes CRUD

Voici l'implémentation complète de `GestionProduitImpJpa` avec les deux attributs JPA et toutes les opérations CRUD :

4.2 Méthodes CRUD avec JPA

Étape 2 : Implémentation de toutes les méthodes CRUD

Voici l'implémentation complète de `GestionProduitImpJpa` :

```
1 package com.poly.jee.dao;
2
3 import java.util.List;
4 import javax.persistence.*;
5 import com.poly.jee.entities.Produit;
6
7 public class GestionProduitImpJpa implements IGestionProduit {
8
9     private EntityManagerFactory emf =
10         Persistence.createEntityManagerFactory("punit1");
11     private EntityManager em = emf.createEntityManager();
12 }
```

```
13 // ---- CREATE ----
14 @Override
15 public void addProduit(Produit p) {
16     EntityTransaction tx = em.getTransaction();
17     try {
18         tx.begin();
19         em.persist(p);    // INSERT INTO produit ...
20         tx.commit();
21     } catch (Exception e) {
22         tx.rollback();    // Annulation en cas d'erreur
23         e.printStackTrace();
24     }
25 }
26
27 // ---- READ (tous les produits) ----
28 @Override
29 public List<Produit> getAllProduits() {
30     // JPQL : langage de requete oriente objet (comme SQL
31     //      mais sur les entites)
32     return em.createQuery("FROM Produit", Produit.class)
33         .getResultList();
34 }
35
36 // ---- READ (par id) ----
37 @Override
38 public Produit getProductById(Long id) {
39     return em.find(Produit.class, id);    // SELECT * WHERE id
40     //      = ?
41 }
42
43 // ---- READ (recherche par nom) ----
44 @Override
45 public List<Produit> searchProduits(String motCle) {
46     return em.createQuery(
47         "FROM Produit p WHERE p.nom LIKE :mc", Produit.class
48     )
49     .setParameter("mc", "%" + motCle + "%")
50     .getResultList();
51 }
52
53 // ---- UPDATE ----
54 @Override
55 public void updateProduit(Produit p) {
56     EntityTransaction tx = em.getTransaction();
57     try {
58         tx.begin();
59         em.merge(p);    // UPDATE produit SET ...
60         tx.commit();
61     } catch (Exception e) {
62         tx.rollback();
63     }
64 }
```

```

60         e.printStackTrace();
61     }
62 }
63
64 // ---- DELETE ----
65 @Override
66 public void deleteProduit(Long id) {
67     EntityTransaction tx = em.getTransaction();
68     try {
69         tx.begin();
70         Produit p = em.find(Produit.class, id);
71         if (p != null) {
72             em.remove(p); //DELETE FROM produit WHERE id = ?
73         }
74         tx.commit();
75     } catch (Exception e) {
76         tx.rollback();
77         e.printStackTrace();
78     }
79 }
80 }

```

Listing 5 – GestionProduitImpJpa.java – CRUD complet avec JPA

Remarque

Rôle de EntityTransaction :

Toutes les opérations d'écriture (`persist`, `merge`, `remove`) doivent être encadrées dans une transaction. Le principe est **tout ou rien** : soit toutes les opérations réussissent (`commit`), soit toutes sont annulées (`rollback`) en cas d'erreur.

Méthode JPA	Opération SQL	Description
<code>em.persist(p)</code>	INSERT	Persiste un nouvel objet
<code>em.find(T, id)</code>	SELECT WHERE id	Recherche par clé primaire
<code>em.merge(p)</code>	UPDATE	Met à jour une entité existante
<code>em.remove(p)</code>	DELETE	Supprime une entité
<code>em.createQuery(...)</code>	SELECT (JPQL)	Requête personnalisée

4.3 Mise à jour de la Servlet

Étape 3 : Modification de la méthode init() de la Servlet

Remplacez l'ancienne implémentation JDBC par l'implémentation JPA dans la Servlet :

```
1 package com.poly.jee.web;
2
3 import com.poly.jee.dao.GestionProduitImpJpa;
4 import com.poly.jee.dao.IGestionProduit;
5 // ... autres imports
6
7 public class FirstServlet extends HttpServlet {
8
9     private IGestionProduit gestion;
10
11     @Override
12     public void init() throws ServletException {
13         // Ancienne version JDBC :
14         // gestion = new GestionProduitImpJdbc();
15
16         // Nouvelle version JPA/Hibernate :
17         gestion = new GestionProduitImpJpa(); // Seul changement
18         !
19     }
20
21     // Le reste de la Servlet est inchangé
22     // grâce au polymorphisme et à l'interface IGestionProduit
23     // ...
24 }
```

Listing 6 – Modification de FirstServlet.java – méthode init()

Attention

Principe ouvert/fermé (Open/Closed Principle) : Grâce à l'interface `IGestionProduit`, seule la ligne d'instanciation change. Tout le code de la Servlet (`doGet`, `doPost`) reste inchangé. C'est l'un des grands avantages de la programmation par interface.

5 Vérification et Tests

Objectif de la section

Valider que l'application fonctionne correctement avec JPA/Hibernate et vérifier la génération automatique de la table en base de données.

5.1 Génération automatique de la table

Étape 1 : Vérification dans la console Eclipse

Lors du premier lancement, Hibernate affiche dans la console le script SQL de création de la table :

```
1 Hibernate:
2   create table produit (
3       id bigint not null auto_increment,
4       nom varchar(255),
5       prix double precision not null,
6       quantite integer not null,
7       primary key (id)
8   ) engine=InnoDB
```

Listing 7 – SQL généré automatiquement par Hibernate (console Eclipse)

Remarque

Cette sortie console confirme qu’Hibernate a bien :

- Lu les annotations `@Entity`, `@Id`, `@GeneratedValue`
- Généré le DDL SQL correspondant
- Créé la table `produit` dans la base `gestioncommv2`

Le fichier `persistence.xml` remplace entièrement l’ancienne classe `SingletonConnection.java`.

5.2 Tests des opérations CRUD

Étape 2 : Réexécution de l’application

Réexécutez l’application sur le serveur Tomcat et vérifiez que toutes les fonctionnalités du TP précédent fonctionnent toujours :

- Affichage de la liste des produits (via Hibernate `SELECT`)
- Ajout d’un nouveau produit (via Hibernate `INSERT`)
- Modification d’un produit (via Hibernate `UPDATE`)
- Suppression d’un produit (via Hibernate `DELETE`)
- Recherche par mot-clé (via JPQL `LIKE`)